

Blog Platform myUnila — Deployment Guide VM Baru (vm9-blog)

Versi: 1.0 **Tanggal:** 2026-05-18 **Target:** vm9-blog.lan.unila.ac.id (192.168.120.49) — Ubuntu 24.04 LTS
Scope: Backend (blog-service) + Frontend public (frontend-blog) + Frontend dashboard di MyUnila (sudah deploy di vm5).

Ringkasan Komponen yang Akan Di-Deploy

Komponen	Lokasi	Port	Image
Nginx (reverse proxy + SSL)	vm9-blog	80 / 443	nginx:alpine
Frontend-blog (Next.js)	vm9-blog	127.0.0.1:3002	myunila-frontend-blog:latest
Blog-service (Go + Fiber)	vm9-blog	127.0.0.1:8091	myunila-blog-service:latest
PostgreSQL 16 (blog_unila)	vm9-blog	127.0.0.1:5432	postgres:16-alpine
MinIO (storage)	vm5 atau dedicated	9000	(sudah ada)
Meilisearch (search)	vm2 atau dedicated	7700	(sudah ada, shared)
SMTP relay	Unila central	25/587	(eksternal)
Frontend dashboard	vm5 (myunila.unila.ac.id)	(existing)	(perlu config saja)

Tidak perlu service baru di vm5 — dashboard `/dashboard/blog-platform/*` sudah built-in di frontend MyUnila. Yang perlu di vm5 hanya rebuild dengan env var baru yang menunjuk ke API blog-service.

Status Code per 2026-05-18

- ✅ **Done dan ter-push ke bitbucket master:** - Sprint 11-12 (Phase R-AY, 32 phase) - Sprint 13 (BF, BE, BD, BA, BB + Quick Wins) - Integration tests untuk 3 critical flow (BB, BD, BF) — 24 test pass
- ⚠️ **Belum di-tune-up secara formal** (penting untuk Anda tahu sebelum prod): - Load test (k6/wrk) — belum dijalankan, traffic profile prod belum di-measure - Lighthouse audit frontend-blog — belum, score belum diketahui - Image optimization (CDN, lazy load, WebP) — pakai default Next.js - Database query plan analysis di prod-scale data — belum - Rate limiting per-endpoint di Kong — pakai default - CSP / HSTS header strict — basic, belum hardened
- 🛑 **Blocked / Deferred:** - Phase AZ (plagiarism check) — butuh procurement API Turnitin/Plagscan - Phase BC (static export) — niche, deferred indefinitely

Bagian 1 — Pre-Deploy Checklist

Pastikan sebelum mulai:

1.1 Infra ready

- [] VM vm9-blog provisioned (Ubuntu 24.04, user `myblog` , sudo + docker group)
- [] DNS A record:
 - `blog.unila.ac.id` → 192.168.120.49
 - `*.blog.unila.ac.id` → 192.168.120.49 (wildcard, untuk subdomain per-user)
 - `api.blog.unila.ac.id` → 192.168.120.49 (kalau pakai subdomain API terpisah)
- [] SSL wildcard certificate Let's Encrypt sudah issued (DNS-01 via Cloudflare)
- [] Firewall buka 80, 443 inbound dari publik; 5432, 6379, 8091, 3002 closed dari publik
- [] Akses SSH dari laptop dev ke vm9-blog OK
- [] Docker + Docker Compose v2 sudah terinstall di VM

1.2 Secrets & credentials

- [] JWT_SECRET — sama dengan auth-service vm2 (Kong-trust pattern)
- [] Postgres password vm9 (boleh beda dari vm lain — local-only access)
- [] MinIO access/secret key (production cluster — minta tim infra)
- [] Meilisearch master key + read-only search key
- [] SMTP credentials Unila central relay (host, port, username, password)
- [] VAPID keypair web push — **belum digenerate** (langkah di bawah)

1.3 Code ready

- [x] Branch `master` bitbucket terbaru includes Sprint 13 (commit `b642894bd` atau setelahnya)
 - [x] 5 alter Sprint 13 sudah baked di `01-blog_unila_v1.0_fresh.sql` (fresh install ambil ini langsung)
-

Bagian 2 — Deploy di vm9-blog (Step-by-Step)

Step 1 — SSH + Setup user

```
# Dari Laptop dev:
ssh myblog@192.168.120.49

# Verify docker:
docker --version
docker compose version

# Pastikan user di docker group:
groups | grep -q docker || sudo usermod -aG docker $USER
# (Logout+Login kalau baru ditambahkan)
```

Step 2 — Clone repo

```
# Di vm9-blog:
sudo mkdir -p /var/www
sudo chown myblog:myblog /var/www
cd /var/www

git clone https://bitbucket.org/mahendraunila/my-unila.git
cd my-unila

# Verify branch:
git branch --show-current # harus master
git log -1 --oneline      # cek commit terbaru (paling tidak Sprint 13)
```

Step 3 — Konfigurasi `.env`

```
cd deployment/production/vm9-blog
cp .env.example .env
nano .env # atau vim
```

Isi nilai-nilai berikut (lihat `.env.example` untuk daftar lengkap):

```
# PostgreSQL local di vm9
POSTGRES_PASSWORD=<random_strong_password>

# JWT – HARUS SAMA dengan auth-service vm2
JWT_SECRET=<copy_dari_vm2_auth_service>

# MinIO (production cluster)
MINIO_ENDPOINT=https://minio.unila.ac.id
MINIO_ACCESS_KEY=<from_infra>
MINIO_SECRET_KEY=<from_infra>
MINIO_BUCKET=blog-media

# Meilisearch (shared di vm2)
MEILI_HOST=http://192.168.120.42:7700
MEILI_API_KEY=<master_key>

# SMTP – kosongkan, set via Admin UI nanti
# (tabel blog.mail_config singleton, di-set dari dashboard)

# Web push – generate dulu di Step 5
WEBPUSH_PUBLIC_KEY=
WEBPUSH_PRIVATE_KEY=
WEBPUSH_SUBJECT=mailto:dev@unila.ac.id

# Frontend-blog
NEXT_PUBLIC_BLOG_API=https://api.blog.unila.ac.id
NEXT_PUBLIC_APEX_HOST=blog.unila.ac.id
NEXT_PUBLIC_USE MOCK=false

# Webhook
REVALIDATE_SECRET=$(openssl rand -hex 32)

TZ=Asia/Jakarta
```

Pastikan: `chmod 600 .env` supaya cuma user `myblog` yang bisa baca.

Step 4 — Init Database PostgreSQL

Boot Postgres dulu, lalu apply schema:

```

cd /var/www/my-unila/deployment/production/vm9-blog

# Start cuma postgres dulu
docker compose up -d postgres
sleep 5

# Verify connect
docker compose exec postgres pg_isready -U postgres

# Buat database + schema awal
docker compose exec -T postgres psql -U postgres -c "CREATE DATABASE blog_unila;"

# Apply fresh.sql (sudah includes semua Sprint 11-13 alter)
docker compose exec -T postgres psql -U postgres -d blog_unila \
< /var/www/my-unila/data-model/script/postgresql/blog/01-blog_unila_v1.0_fresh.sql

# Apply seed data referensi (kategori, tag awal, dst)
docker compose exec -T postgres psql -U postgres -d blog_unila \
< /var/www/my-unila/data-model/script/postgresql/blog/02-blog_unila_v1.0_seed.sql

# Verify tabel ter-create
docker compose exec postgres psql -U postgres -d blog_unila -c "\dt blog.*; \dt interaction.*; \dt m

```

Pastikan ke-22 tabel (17 base + 5 alter Sprint 13) muncul.

Step 5 — Generate VAPID Keys (Web Push, Phase BA)

```

cd /var/www/my-unila/backend/blog-service

# Build sekali untuk dapat binary (atau pakai go run langsung)
docker run --rm -v $(pwd):/src -w /src golang:1.24-alpine \
go run ./tools/gen-vapid

# Output 3 baris ENV — copy ke .env vm9-blog

```

Update `.env` dengan WEBPUSH_PUBLIC_KEY + PRIVATE_KEY + SUBJECT.

PENTING: simpan keypair ini terpisah di password manager. Kalau hilang, semua user yang sudah subscribe push notification harus re-subscribe (karena keys lama gak match).

Step 6 — Build + Start semua container

```
cd /var/www/my-unila/deployment/production/vm9-blog

# Build images (pertama kali, akan agak lama – ~5-10 menit)
docker compose build --no-cache

# Start semua service
docker compose up -d

# Verify
docker compose ps
# semua harus "Up" + healthy untuk blog-service, postgres
```

Log check:

```
docker compose logs -f blog-service | head -30
# Cari baris:
#   ✓ Connected to PostgreSQL
#   ✓ MinIO connected
#   ✓ Meilisearch connected
#   📢 Web push enabled (VAPID public key set)
#   📧 Email outbox worker started
#   📧 Weekly digest worker started
#   📅 Post scheduler started
#   📈 Trending score scheduler started
#   ✓ Listening on :8091
```

Step 7 — SSL + Nginx config

```
cd /var/www/my-unila/deployment/production/vm9-blog/configs/nginx
# Verify nginx config sudah point ke domain yang benar
nano conf.d/blog.conf
# Pastikan server_name = blog.unila.ac.id + *.blog.unila.ac.id

# Reload nginx
docker compose exec nginx nginx -t      # test config
docker compose exec nginx nginx -s reload
```

SSL certificate path di nginx config harus point ke `/etc/letsencrypt/live/blog.unila.ac.id/`. Mount volume Let's Encrypt sudah di docker-compose.yml.

Step 8 — MinIO bucket setup

```
# Dari laptop dev atau dari vm9, asalkan ada mc client:
mc alias set unila-prod https://minio.unila.ac.id <ACCESS_KEY> <SECRET_KEY>

# Buat bucket
mc mb unila-prod/blog-media

# Set bucket policy public read (file dipublish bisa diakses tanpa auth)
mc anonymous set download unila-prod/blog-media

# Verify
mc ls unila-prod/blog-media
```

Step 9 — Meilisearch index setup

```
# Test connect
curl -X GET 'http://192.168.120.42:7700/health' -H "Authorization: Bearer $MEILI_API_KEY"

# Create index 'blog_post' (kalau belum ada)
curl -X POST 'http://192.168.120.42:7700/indexes' \
  -H "Authorization: Bearer $MEILI_API_KEY" \
  -H 'Content-Type: application/json' \
  -d '{"uid": "blog_post", "primaryKey": "id_post"}'

# Trigger full resync dari blog-service (dia auto sync, atau via admin endpoint)
# blog-service akan auto-trigger search.FullResync di startup – cek log:
docker compose logs blog-service | grep "search.FullResync"
```

Step 10 — SMTP config via Admin UI (Phase AY)

Akses dari browser (setelah Bagian 3 frontend dashboard juga deploy):

1. Login sebagai admin di `https://myunila.unila.ac.id`
 2. Navigasi ke `/dashboard/blog-platform/admin/mail-config`
 3. Klik “Tambah Profile”: - Label: `Unila SMTP Production` - Host: SMTP relay Unila (minta tim infra) - Port: 587 (atau 465 kalau implicit TLS) - Username + Password - TLS: ON, STARTTLS: ON (sesuai relay) - From: `blog-noreply@unila.ac.id` - Public URL: `https://blog.unila.ac.id`
 4. Klik “Aktifkan” — singleton invariant enforce 1 profile aktif
 5. Test send ke `dev@unila.ac.id` — verifikasi email masuk
-

Bagian 3 — Config Frontend MyUnila (dashboard di vm5)

Dashboard `/dashboard/blog-platform/*` sudah ada di frontend MyUnila. Yang perlu di-update di vm5:

3.1 Env vars

Edit `.env.production` di frontend MyUnila (vm5):

```
# API endpoint untuk blog-service (via Kong atau langsung)
NEXT_PUBLIC_BLOG_API=https://api.blog.unila.ac.id
# atau via Kong gateway:
# NEXT_PUBLIC_BLOG_API=https://api.unila.ac.id/blog-service

# Apex host (untuk link "Lihat blog kamu" di dashboard)
NEXT_PUBLIC_BLOG_APEX_HOST=blog.unila.ac.id

# Kalau pakai web push (Phase BA) di dashboard:
# Frontend ambil VAPID public key via GET /api/v1/push/vapid-public,
# JADI tidak perlu set NEXT_PUBLIC_VAPID_PUBLIC_KEY di env.
# Service worker file blog-push-sw.js sudah di /frontend/public/.
```

3.2 Kong route (kalau pakai gateway)

Tambah route di Kong config:

```
- name: blog-service
  url: http://192.168.120.49:8091
  routes:
    - paths: ["/blog-service"]
      strip_path: true
      preserve_host: false
```

3.3 Portal apps seeder

Sudah ada di seeders Sprint 13 ([backend/auth-service/database/seeders/data/portal_menus/blog-platform.json](#)). Jalankan di auth-service vm2:

```
docker exec myunila-auth-service-staging \
  php artisan db:seed --class=PortalAplikasiSeeder
```

Verifikasi: - Portal "Blog Unila" muncul di `/portal-apps` MyUnila - Author punya akses ke `/dashboard/blog-platform/` - Admin (developer/admin/blog_admin role) punya akses ke `/dashboard/blog-platform/admin/*`

3.4 Rebuild + restart frontend MyUnila

```
# Di vm5:
cd /var/www/my-unila/deployment/production/vm5-staging
./scripts/rebuild-service.sh frontend
docker logs myunila-frontend-staging --tail 30
```

Bagian 4 — Smoke Test Production

Pasca-deploy, verifikasi flow critical:

4.1 Public surfaces

Test	URL	Expect
Apex homepage	https://blog.unila.ac.id	List trending/latest, kategori, top authors
Per-tenant blog	https://bambang-dosen.blog.unila.ac.id	Profile + post list
Public post	https://bambang-dosen.blog.unila.ac.id/posts/judul-slug	Konten + komentar + subscribe form
RSS feed	https://blog.unila.ac.id/rss	Valid XML
Sitemap	https://blog.unila.ac.id/sitemap.xml	List URL
OG image	https://blog.unila.ac.id/api/v1/og?title=Test	PNG 1200×630
VAPID public key	https://api.blog.unila.ac.id/api/v1/push/vapid-public	JSON with public_key
Search autocomplete	https://blog.unila.ac.id/search?q=nextjs	Suggestions appear

4.2 Dashboard flow

1. Login ke MyUnila → portal apps → Blog Unila
2. Klaim subdomain (kalau belum punya blog)
3. Tulis post baru → publish → cek muncul di public tenant page
4. Buka post → klik subscribe form (di tenant page) → email konfirmasi datang
5. Konfirmasi email → status confirmed di dashboard `/subscribers`
6. Notifikasi: enable push → klik Test → notif muncul di browser
7. Bilingual: bikin post EN → link pair ke post ID → buka public post, toggle bahasa works

4.3 Worker check

Tunggu 15-30 menit setelah deploy, cek log:

```
docker compose logs blog-service --since 30m | grep -E "trending recomputed|outbox drain|digest tick"
```

Harus ada: -  trending recomputed for N posts (setiap 15 menit) -  outbox drain: X sent, Y failed of Z claimed (kalau ada email queue) -  digest tick: (setiap 6 jam — kalau ada candidate) -  search.FullResync: indexed N posts (saat startup)

Bagian 5 — Recommended Tuneup (PR Terpisah, BUKAN Untuk MVP Launch)

Ini saran improvement post-launch. Belum dijalankan, tidak block launch.

5.1 Performance

- [] Load test pakai k6/wrk untuk endpoint hot path: `/api/v1/posts` , `/api/v1/me/notifications` , `search autocomplete`
- [] Lighthouse audit frontend-blog — target score: Performance ≥ 90 , SEO ≥ 95 , Accessibility ≥ 95
- [] Image optimization: serve WebP/AVIF via Next.js Image + CDN cache header
- [] EXPLAIN ANALYZE query plan untuk `getTrending` , `getRelatedPosts` , `getTopAuthors` di prod-scale data
- [] Cache popular endpoints di Nginx (1-5 menit) atau di Redis

5.2 Security

- [] Rate limit per-endpoint di Kong: subscribe form (10 req/menit per IP), search (60/menit), comment (5/menit)
- [] CSP header strict — saat ini basic. Add `default-src 'self'; img-src 'self' minio.unila.ac.id ui-avatars.com;`
- [] HSTS preload submission (`includeSubDomains`; `preload`)
- [] XSS audit TipTap output — saat ini pakai DOMPurify default, audit allowed tags
- [] Secret scan repo (gitleaks/trufflehog) — pastikan tidak ada hardcoded secret yang ke-commit
- [] WAF basic rules di Cloudflare (kalau pakai)

5.3 Reliability

- [] Backup harian Postgres `blog_unila` → S3/MinIO retention 30 hari
- [] Backup MinIO `blog-media` → secondary bucket retention 30 hari
- [] Healthcheck endpoint di Kong → trigger alert (Telegram/Slack) kalau down >5 menit
- [] Monitoring Grafana — query rate, error rate, response time p95/p99
- [] Log aggregation (Loki / Elasticsearch) — saat ini cuma docker logs

5.4 Observability

- [] Tracing (OpenTelemetry) untuk hot paths
- [] Sentry untuk frontend error tracking
- [] Email outbox alert: kalau `failed_count` >10 dalam 1 jam, page admin
- [] Digest worker alert: kalau `users_due_now` >100 dan worker idle, investigate

Bagian 6 — Rollback Plan

Kalau ada masalah pasca-deploy:

6.1 Rollback container

```
cd /var/www/my-unila/deployment/production/vm9-blog
docker compose down

# Checkout commit sebelum Sprint 13 (kalau perlu)
git checkout 1d49b8cc0 # Last commit before Sprint 13 push

docker compose build --no-cache
docker compose up -d
```

6.2 Rollback database

Sprint 13 alter scripts **additive only** — gak ada DROP TABLE atau breaking constraint. Aman di-skip kalau perlu downgrade aplikasi.

Kalau perlu manual revert (rare):

```
-- Phase BB
DROP TABLE IF EXISTS interaction.subscriber;

-- Phase BA
DROP TABLE IF EXISTS interaction.push_subscription;

-- Phase BD
ALTER TABLE blog.post DROP CONSTRAINT IF EXISTS chk_post_bahasa;
ALTER TABLE blog.post DROP COLUMN IF EXISTS id_pair_post;

-- Phase BE
DROP TABLE IF EXISTS interaction.digest_log;

-- Phase BF
DROP TABLE IF EXISTS blog.banned_commenter;
```

6.3 Rollback frontend dashboard

```
# Di vm5:
cd /var/www/my-unila
git checkout <previous-commit>
cd deployment/production/vm5-staging
./scripts/rebuild-service.sh frontend
```

Bagian 7 — Checklist Final Sebelum Go-Live

- [] Semua Step 1-10 di Bagian 2 selesai
- [] Smoke test public surfaces (Bagian 4.1) pass
- [] Smoke test dashboard flow (Bagian 4.2) pass
- [] Worker log (Bagian 4.3) menunjukkan goroutine jalan
- [] SSL Grade A di SSL Labs (<https://www.ssllabs.com/ssltest/>)
- [] DNS resolve correct dari publik (cek dari dig)
- [] Backup pertama Postgres + MinIO sudah dijalankan
- [] Tim Unila (admin) sudah training pakai dashboard
- [] Komunikasi launch ke civitas (mahasiswa, dosen, staf)

Bagian 8 — Maintenance Routine

Daily (otomatis via cron / systemd timer)

- Backup Postgres ke offsite storage

- Backup MinIO ke secondary bucket
- Rotasi log Docker (`docker logs truncate`)

Weekly

- Review email outbox: `SELECT status, COUNT(*) FROM interaction.email_outbox GROUP BY 1;`
- Review banned commenter: pastikan tidak ada false-positive
- Review subscriber confirmed_count growth — track engagement metric

Monthly

- Update OS packages: `sudo apt update && sudo apt upgrade -y`
- Update Docker images (re-pull base images, rebuild)
- Renew SSL certificate (auto via acme.sh, verify)
- Vacuum + analyze Postgres: `docker compose exec postgres vacuumdb -U postgres -d blog_unila --analyze --verbose`

Quarterly

- Performance audit (Lighthouse, k6)
- Security audit (manual review + automated scan)
- Backup restore test (verify backup actually works)

Lampiran A — Daftar Lengkap Env Var Production

Lihat [deployment/production/vm9-blog/.env.example](#) untuk template. Ringkasnya:

Env var	Lokasi	Wajib	Catatan
POSTGRES_PASSWORD	vm9 docker-compose	✓	Random strong
JWT_SECRET	vm9 + vm2 (sync)	✓	Sama dengan auth-service
MINIO_ACCESS_KEY / _SECRET_KEY	vm9	✓	Dari infra
MINIO_BUCKET	vm9	✓	blog-media
MEILI_HOST	vm9	✓	vm2:7700
MEILI_API_KEY	vm9	✓	Master key
WEBPUSH_PUBLIC_KEY / _PRIVATE_KEY	vm9	⚠	Empty = push disabled (degrade gracefully)
WEBPUSH_SUBJECT	vm9	⚠	mailto:dev@unila.ac.id
REVALIDATE_SECRET	vm9	✓	Random 64 hex
NEXT_PUBLIC_BLOG_API	vm9 frontend-blog	✓	https://api.blog.unila.ac.id
NEXT_PUBLIC_APEX_HOST	vm9 + vm5	✓	blog.unila.ac.id
NEXT_PUBLIC_USE MOCK	vm9 frontend-blog	✓	false di prod

Lampiran B — Endpoint API Lengkap

Total 246 fiber handler. Highlights per kategori:

Kategori	Path prefix	Count
Public post	/api/v1/posts/*	~20
Public blog	/api/v1/blogs/*	~10
Public search	/api/v1/search/*	~5
Public OG / RSS / sitemap	/api/v1/og , /rss , /sitemap	~5
Public push key	/api/v1/push/vapid-public	1
Public subscribe (BB)	/api/v1/blogs/.../subscribe , /confirm/:token , /unsubscribe/:token	3
Public komentar	/api/v1/posts/:id/komentar	2
Public like/bookmark	/api/v1/posts/:id/like , /bookmark	4
Authenticated /me/blog/*	post, komentar, follower, media, series, co-author, etc.	~150
Authenticated /me/notifications/*	feed, preferences, push subscribe (BA)	~15
Admin /admin/*	curation, banned user, mail config, reserved words	~30

Detail lengkap: lihat `docs/blog-platform/03-api-endpoints.md` (perlu di-update untuk Sprint 13 endpoints).

Lampiran C — Troubleshooting Cepat

Gejala	Kemungkinan penyebab	Fix
Email tidak terkirim	Mail config belum diaktifkan	Cek <code>/dashboard/blog-platform/admin/mail-config</code> → ada profile aktif?
Web push gagal subscribe	VAPID key empty	Cek <code>docker logs blog-service \</code> <code>grep "Web push"</code> → harus "enabled"
Search hasil kosong	Index belum di-resync	<code>docker logs blog-service \</code> <code>grep FullResync</code> — atau restart blog-service untuk trigger
Foto upload gagal	MinIO bucket policy / credentials	Cek <code>mc ls unila-prod/blog-media</code> dari vm9
Subdomain blog 404	DNS wildcard belum propagate	<code>dig <sub>.blog.unila.ac.id</code> — pastikan resolve ke vm9 IP
Blog-service unhealthy	Database connection lost	<code>docker compose restart blog-service</code> + cek Postgres log
Frontend slow	Next.js cache cold	Cek Nginx access log — kalau banyak <code>MISS</code> , tune <code>revalidate</code>
Trending score stuck di 0	Worker belum tick	Tunggu 15 menit atau restart blog-service

Dokumen ini bersifat hidup — update setiap kali ada perubahan signifikan deployment. Versi: 1.0 (2026-05-18).